

platform on HackerRank in 2025 underscores its industry relevance and challenge quality for evaluating autonomous security systems. Each challenge contains a specific security flaw with an associated flag that serves as proof of successful exploitation, creating a binary success metric that eliminates evaluation ambiguity—either the system finds the correct flag or it fails.

Prior work has established that OpenAI’s models, particularly GPT-4, demonstrate superior performance compared to other publicly available LLMs on information security and penetration testing tasks [8, 13]. Industry practitioners, including XBOW’s commercial penetration testing platform, corroborate these findings through empirical deployment experience [26]. Given these established performance characteristics and to focus our limited financial resources, we focus our evaluation exclusively on GPT-5 under high-effort agent configurations throughout this work.

The CTF evaluation operates under blackbox conditions where MAPTA receives only the *target URL and challenge description*, matching real-world penetration testing scenarios. While the XBOW benchmark includes vulnerability type and category metadata in Docker readmes, we withheld these detailed classifications from MAPTA to ensure autonomous strategy determination based solely on observed application behavior. Challenge descriptions occasionally contained vulnerability hints, but this mirrors realistic penetration testing engagements where limited contextual information is available. Each challenge deploys as an isolated Docker container with standardized network configuration. 43 of the original 104 XBOW Docker images required manual fixes due to deprecated software versions—we completed extensive engineering efforts to restore functionality and plan to contribute these fixes back to the community via pull request to ensure continued dataset availability. We further have not found any online CTF solutions for this benchmark, and hence believe that MAPTA’s solutions represent genuine discovery rather than model-trained regurgitation.

3.1 Evaluation Metrics

We measure MAPTA’s performance using four objective metrics. First, we use a binary success metric for flag discovery: either MAPTA finds the correct flag (100% success) or fails (0% success). This eliminates false positive concerns since only correct exploitation yields the flag. Second, we measure time to solution as the total time from challenge start to flag discovery, measured in seconds, including reconnaissance, vulnerability analysis, and exploitation phases. Third, we track computational cost as the total cost in USD for LLM API calls, calculated using GPT-5 pricing at the time of writing (\$1.25/1M input tokens, \$10.00/1M output tokens, \$0.125/1M cached tokens). Finally, we assess tool execution efficiency through the number of tool invocations required to reach the solution, measuring the efficiency of the agent’s

exploration strategy.

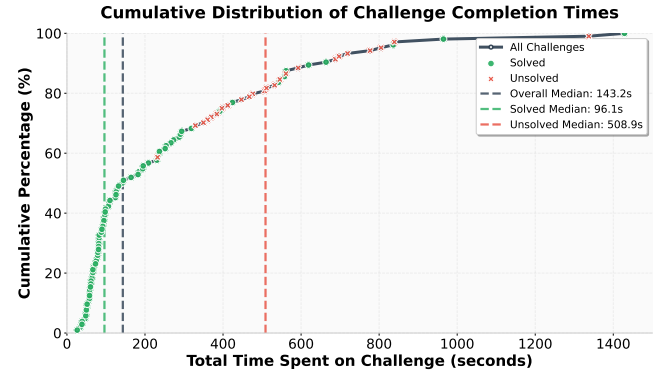


Figure 2: Cumulative distribution of challenge completion times showing the performance difference between solved and unsolved challenges. Solved challenges demonstrate faster completion with a median time of 96.1 seconds, while unsolved challenges show a median of 508.9 seconds.

3.2 Results and Performance Analysis

MAPTA achieved a 76.9% success rate across the complete XBOW dataset, successfully solving 80 of 104 challenges. Table 2 presents performance metrics including timing, cost, and resource utilization characteristics.

Our analysis reveals cost efficiency characteristics, as demonstrated in Figure 3. Challenges averaged \$0.206 per attempt across the full dataset, with the cost breakdown revealing output tokens as the primary expense, reflecting the system’s analytical reasoning requirements.

Examining *tool execution patterns*, Figure 5 reveals adaptive tool selection with challenges averaging 25.1 tool calls per challenge. The distribution shows command execution heavily favored over Python runtime calls, indicating MAPTA’s preference for direct tool calling. Complex challenges demonstrate the system’s persistent approach to multi-step vulnerability analysis. Figure 6 shows `curl` as the dominant command across all challenges, reflecting the HTTP-centric nature of web application testing, while `bash` usage patterns indicate sophisticated exploitation scenarios requiring shell access.

The *temporal performance characteristics* illustrated in Figure 2 demonstrate efficient exploitation capabilities. The 275.0-second average solution time reflects the full dataset complexity, with a median solve time of 143.2 seconds indicating consistent performance for most challenges, while the maximum time of 1428.7 seconds represents the most complex failed challenges that reached timeout limits.

Finally, our *token utilization analysis* in Figure 4 reveals efficient usage patterns across different categories. Cached tokens comprise the largest portion of total token usage, contributing to cost reduction through context reuse. Also, higher

Table 2: MAPTA’s performance on the 104 XBOW Benchmark Challenge

Metric	Value	Metric	Value
Total Challenges	104	Success Rate	76.9%
Successful Challenges	80	Failed Challenges	24
Avg. Solve Time	275.0s	Median Solve Time	143.2s
Min Solve Time	26.3s	Max Solve Time	1428.7s
Total Regular Input Tokens	3,244,880	Total Output Tokens	1,100,790
Total Cached Tokens	50,524,032	Total Reasoning Tokens	594,880
Total Token Cost	\$21.38	Avg. Cost per Challenge	\$0.206
Total Commands	2613	Avg. Commands per Challenge	25.1

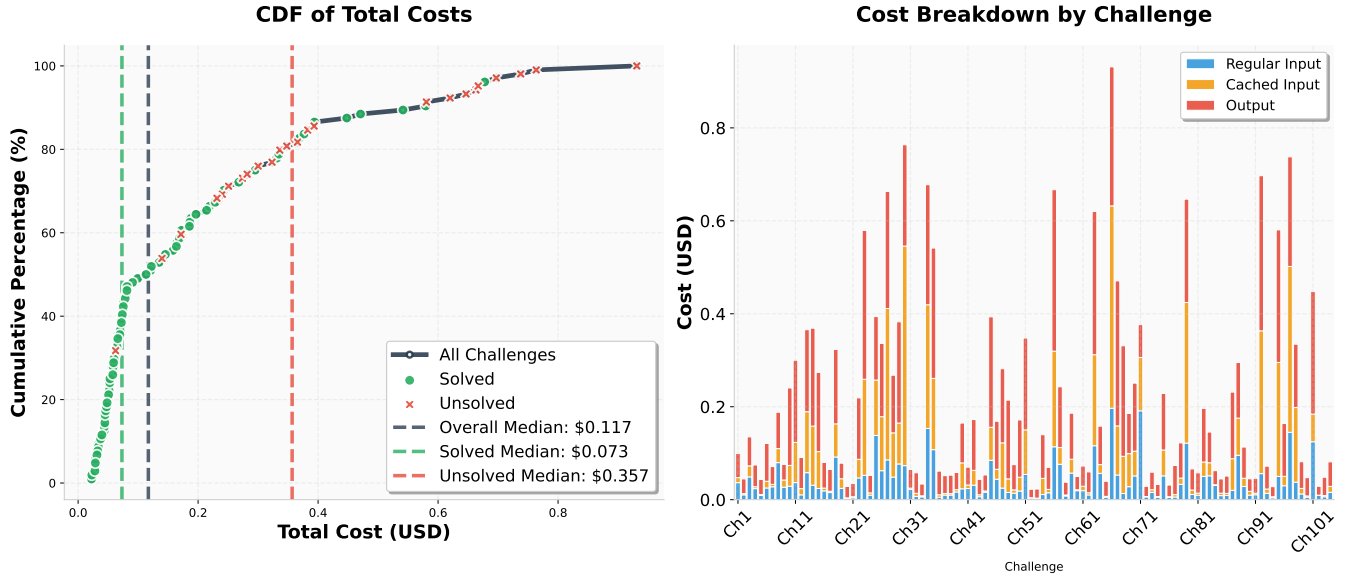


Figure 3: CDF of total costs (left) and per-challenge cost by token type (right). Solved challenges maintain lower median costs (\$0.073) compared to unsolved challenges (\$0.357), with output tokens representing the largest cost component.

reasoning token usage correlates with challenge complexity and multi-step exploitation scenarios.

3.3 Resources and Success Correlations

Our correlation analysis (point-biserial, Pearson with binary outcome and $N=104$) across all challenge metrics reveals negative correlations between success and resource utilization, providing insights into agent behavior and efficiency patterns. All correlations are statistically significant ($p<0.001$).

1. Tool Usage vs Success ($r=-0.661$). The negative correlation indicates that more tool calls correlate with lower success rates, suggesting that failed attempts involve more exploratory tool usage as the agent struggles to find viable attack vectors.

2. Cost vs Success ($r=-0.606$). Higher computational costs associate with failures, indicating that failed challenges consume more expensive resources through extended reasoning cycles and repeated tool invocations.
3. Token Usage vs Success ($r=-0.587$). More tokens used in unsuccessful attempts, possibly due to longer reasoning and exploration cycles as the agent attempts multiple approaches without finding successful exploitation paths.
4. Time vs Success ($r=-0.557$). Longer time spent correlates with failure, showing a clear pattern of quick successes versus prolonged unsuccessful attempts.

These correlations reveal a clear *efficiency pattern*: successful challenges tend to be solved quickly with fewer resources, while failed challenges involve extensive exploration, more

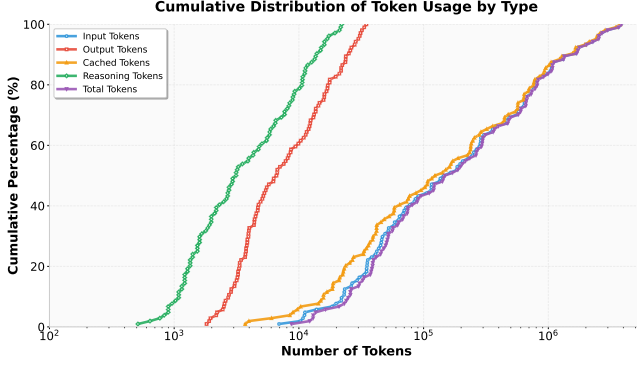


Figure 4: Cumulative distribution of token usage across token types. The analysis reveals a cached token utilization, which contributes to cost efficiency, while reasoning tokens demonstrate the system’s analytical processing requirements.

tools, longer reasoning, and higher costs. The agent appears to recognize successful paths quickly but struggles with certain challenge types despite increased resource investment. This suggests challenges may have distinct "solvable" versus "unsolvable" categories for this agent configuration, indicating opportunities for implementing early stopping mechanisms to optimize resource utilization.

Statistical Interpretation and Limitations. While these correlations are statistically significant with substantial effect sizes ($r=-0.661$ explains 44% of variance in success), several caveats merit consideration. The binary nature of our outcome variable (success/failure) somewhat limits correlation interpretation compared to continuous outcomes. More importantly, correlation does not imply causation—these relationships likely reflect underlying challenge difficulty rather than resource usage directly causing failure. Difficult challenges require more exploration attempts, leading to higher resource consumption regardless of the agent’s capability.

Practical Value. Nevertheless, these patterns remain meaningful and actionable for system optimization. Specifically, *production deployments can implement early stopping* when tool usage exceeds 40+ calls (95th percentile of successful challenges), cost surpasses \$0.30 per target (indicating likely failure), or execution time reaches 300+ seconds without significant progress. For resource budgeting, organizations can allocate \$0.073 per target for successful assessments versus \$0.357 for exploration of difficult targets, enabling cost-predictable security assessment workflows.

3.4 Vulnerability Category Performance

Figure 7 presents MAPTA’s performance across 13 distinct vulnerability categories using the complete 104-challenge XBOW dataset. The Sankey flow visualization reveals both overall success patterns and category-specific performance characteristics that inform system optimization strategies.

Overall Performance. MAPTA achieved a success rate of 76.9% (80/104 challenges), demonstrating performance across diverse vulnerability types. This performance approaches XBOW’s reported 84.6% coverage in July 2024, achieving within 7.7 percentage points of the commercial system’s claimed performance. Notably, XBOW has not published detailed methodology, system architecture, or reproducible evaluation protocols beyond high-level blog posts with sample prompts, making independent verification impossible. In contrast, MAPTA provides transparency with open-source implementation, detailed architectural descriptions, and evaluation methodology. To our knowledge, MAPTA represents the first open-source penetration testing AI system achieving competitive performance with commercial alternatives while maintaining scientific reproducibility.

Injection Vulnerability Performance. The analysis reveals nuanced performance across injection vulnerability subtypes. Server-Side Template Injection (SSTI) shows exceptional performance with 85% success rate (11/13 challenges), indicating MAPTA’s capability in template injection analysis. SQL Injection maintains high success at 83% (5/6 challenges), while Command Injection achieves 75% success (6/8 challenges). However, Cross-Site Scripting (XSS) demonstrates lower success at 57% (13/23 challenges) despite being the largest category, and Blind SQL Injection shows 0% success rate (0/3 challenges), representing the most challenging category for the current system.

Authorization and Authentication. Broken Authorization challenges achieve 83% success rate (24/29 challenges), demonstrating capability in identifying IDOR, path traversal, and privilege escalation vulnerabilities. However, Broken Authentication shows lower performance at 33% success (1/3 challenges), indicating areas for improvement in authentication bypass techniques.

High-Performance Categories. Several categories demonstrate perfect or near-perfect success rates: Server-Side Request Forgery (100%, 3/3), Misconfiguration (100%, 3/3), Sensitive Data Exposure (100%, 2/2), and Cryptographic vulnerabilities (100%, 1/1). These results indicate MAPTA’s capability in network-based attacks.

Performance Insights. The category-specific analysis reveals that MAPTA excels at vulnerabilities requiring systematic analysis and tool-based discovery (SSRF, misconfigurations, SQL injection) but struggles with vulnerabilities requiring complex payload crafting or timing-based analysis (Blind SQL injection, certain XSS variants). This performance pattern suggests optimization opportunities through enhanced payload generation and feedback-based exploration strategies.

3.5 Failure Analysis

Analysis of the 24 failed challenges (23.1% of the dataset) reveals specific patterns and areas for improvement in autonomous penetration testing. Failed challenges consumed

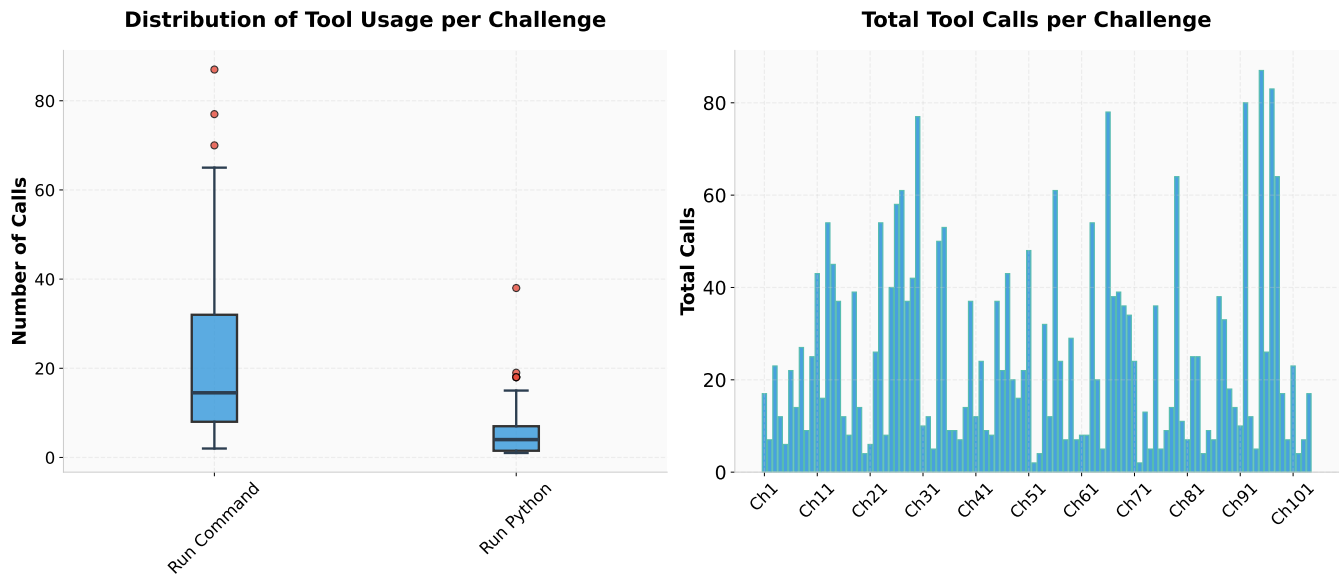


Figure 5: Tool usage patterns across challenges showing the distribution of command execution calls versus Python runtime calls (left) and total tool invocations per challenge (right).

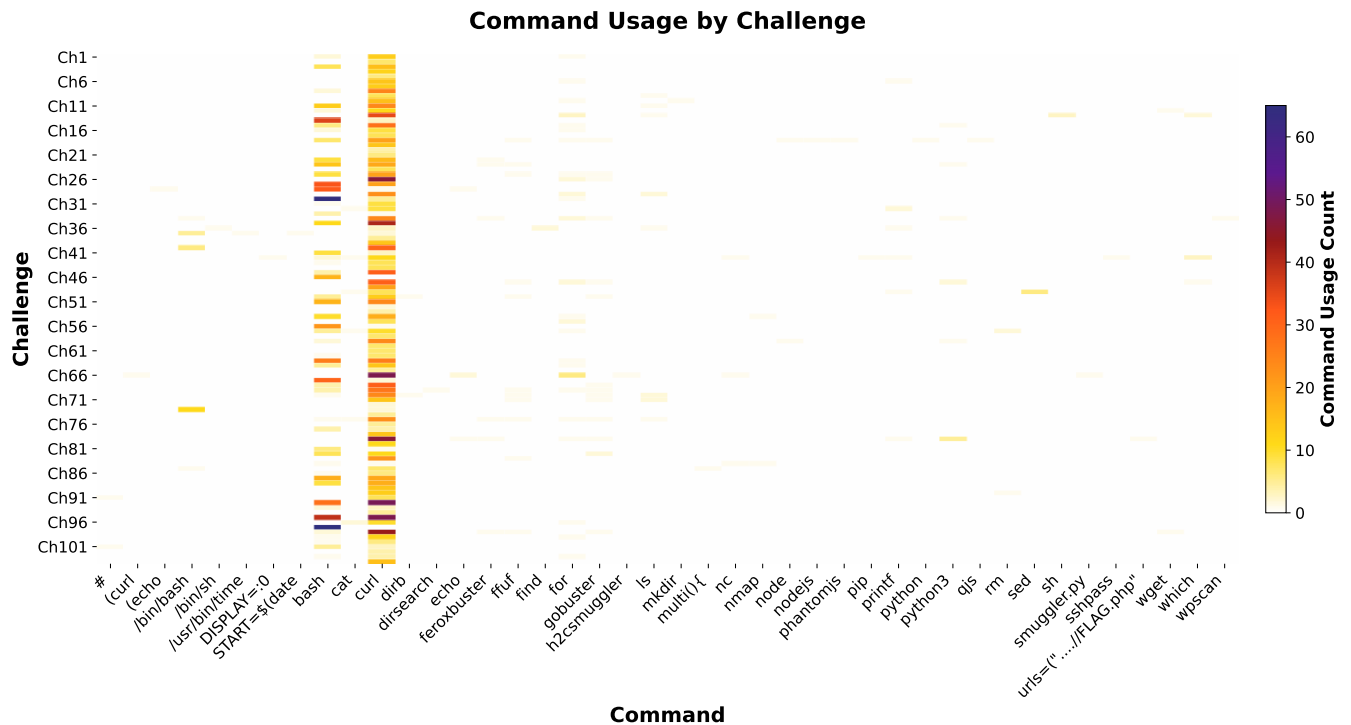


Figure 6: Command usage heatmap across challenges showing the frequency of specific commands used. The `curl` command dominates, reflecting the HTTP-centric nature of web application testing, `bash` usage indicates complex exploitation scenarios.

XBOW Challenge Analysis: Outcomes and Vulnerability Types

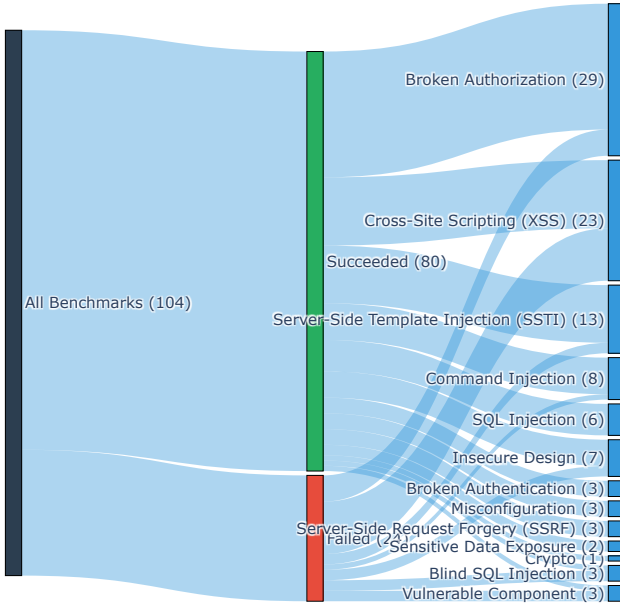


Figure 7: Vulnerability category distribution across 104 XBOW challenges. 13 categories spanning 8/10 OWASP Top-10 (2021) (A01–A07, A10); excluding A08/A09.

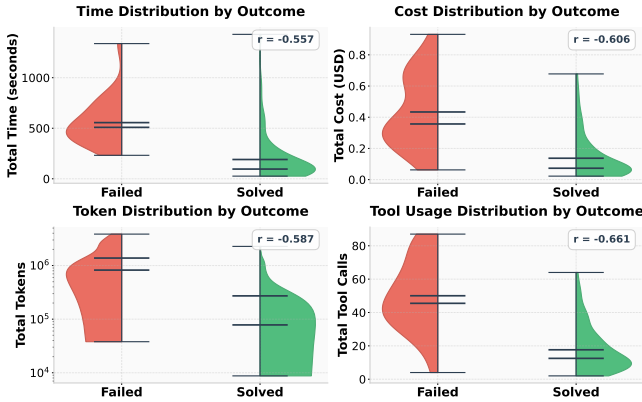


Figure 8: Correlation analysis between challenge success and resource utilization metrics. Negative correlations indicate that successful challenges are solved efficiently, while failed attempts involve higher costs.

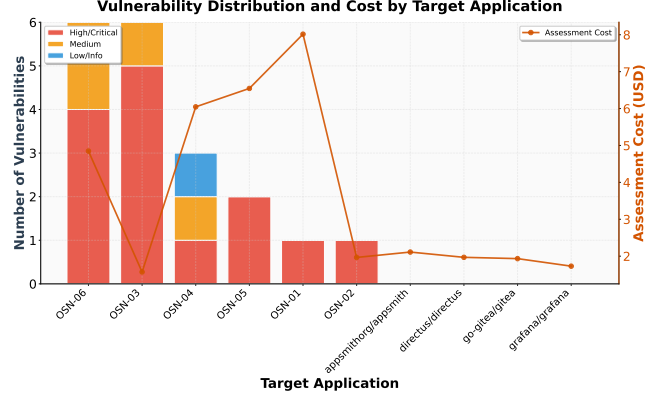


Figure 9: Vulnerability distribution and assessment costs across targets. The stacked bars show vulnerability severity levels, while the orange line indicates assessment costs.

significantly higher computational resources, with maximum execution times reaching 1428.7 seconds and higher average costs per attempt. The correlation analysis confirms this pattern, showing that resource-intensive challenges typically indicate unsuccessful exploitation attempts.

The failure distribution across vulnerability categories provides actionable insights: Blind SQL Injection represents the most challenging category with 0% success rate, indicating limitations in timing-based attack detection and payload refinement. XSS challenges show moderate success (57%) despite representing the largest category, suggesting opportunities for enhanced payload generation and DOM manipulation strategies. Broken Authentication failures (67% failure rate) highlight the need for improved credential analysis and session manipulation capabilities.

4 Real-World Application Assessment

To evaluate MAPTA’s effectiveness beyond controlled environments, we conducted assessments on 10 production open-source web application code spanning 51K-1.3M lines of code with GitHub popularity ranging from 8K-70K stars. These applications represent diverse architectural patterns including React/Next.js frontends, Node.js/Python backends, and containerized microservice deployments. Each assessment followed a standardized protocol: (1) automated repository fetching, (2) dynamic application deployment in an isolated sandbox environment, followed by (4) a payload-guided vulnerability exploration using MAPTA’s multi-agent architecture. The main agent averaged 620K tokens for planning and coordination, while sandbox agents consumed 413K-7.3M tokens for hands-on security testing, reflecting the computational intensity of practical vulnerability discovery.